

Palestine Ahliya University

Faculty of Graduate Studies

Machine Learning

Final Project

Instructor: Mutaz Abu Sara

Student: Andres Emaya

A Robust Machine Learning Pipeline: Regression, Classification and Preprocessing of the Ames Housing Dataset

Abstract

This project is a representation of a Machine-Learning pipeline for predicting house prices. The dataset that will be used is the Ames housing dataset, which is a regression task. We are going to re-frame this task into a classification task in order to compare results with different machine learning models. This is done by converting the Sale Price feature (label) into categories for a classification prediction task. The division of the dataset allows it to support and predict different categories such as affordable, standard, or expensive.

The pipeline that is used in this paper emphasizes the importance of useful pre-processing techniques and different model comparison. The techniques used in this paper include: missing value treatment, scaling, encoding, handling imbalanced data, comparisons between different models, and using some evaluation metrics to show the robustness of the different models that have been used in the experiment. For the numerical values, they are imputed with the median and paired with the RobustScaler. For the categorical values, they are imputed with the mode, and encoded using the one-hot encoder. As for the evaluation, the regression models are evaluated using RMSE, R2, and MAPE. As for the classification models, they are assessed with the accuracy, F1-score, precision, recall, and a confusion matrix. We have found that after splitting the data into classes, there were imbalances, which were fixed with oversampling techniques. It shows how turning a regression task into a classification task can be beneficial.

Keywords

Ames Housing; machine learning; regression; classification; preprocessing; imbalanced data; house-price prediction; Random Forest; Gradient Boosting; one-hot encoding; model evaluation.

Introduction

House-price prediction is a real-world problem with a tangible practical meaning. Buyers want fair estimates. Sellers want to understand the value of their properties. Banks require approximate valuations when making mortgage decisions. Similarly, local governments and analysts may examine property values when planning taxes, infrastructure, or housing policy. The task is not simple. A house price can vary with lot area, living area, age, building quality, neighborhood, basement condition, garage type, sale condition, or numerous other variables. Some of these characteristics are numeric. Others are categorical. Many are incomplete. This makes housing data

a conducive medium for testing an entire machine learning workflow, rather than just training a model on already-clean values. The project for this course uses the Ames Housing dataset and develops two linked tasks. The first task is regression, which is where the model predicts the numerical SalePrice of the house. The second example is classification, where the same target is converted to price classes. This second kind of task expands the scope of the project because there are many others (like class distributions) along with the ability for modeling and imbalance handling also with data which was constructed for regression. The conversion is not viewed as some trick. We treat it as a modelling decision that needs to be explained, verified and assessed. A central point in this work is that preprocessing is not an add-on. The model is an extension of it. A regression or classification algorithm will only accept the data once missing values, scaling and categorical encoding have been processed properly. And if these steps are not made smoothly, a model finished will appear successful and fail next time you give it new data. Thus the code uses scikit-learn pipelines and ColumnTransformer so preprocessing is fit only on the training dataset, and then applied to the test dataset. This design minimizes data leakage and renders the experiment reproducible. The general preprocessing configuration used for regression and classification is the same for both models so that they can be compared fairly. The project tries to also correct for imbalanced data, but carefully. The usual mistake made in student projects is to force artificial imbalance or oversample the data before examining the actual distribution of the classes. This implementation does not face that problem. After transforming the SalePrice into three classes, the code calculates the class counts to calculate a number of classes and an imbalance ratio. Only after the ratio crosses a fixed value does the pipeline utilise imbalance-handling methods i.e., class weights and random oversampling. This gives the workflow more functionality and realism. In reality, imbalance is something to sniff out and handle, not something to fake. We use models that cover simple and more flexible types of machine learning. For regression, the notebook makes comparison between Linear Regression, Ridge Regression, K-Nearest Neighbors, Support Vector Regression, Random Forest, Extra Trees, and Gradient Boosting. For classification it compares Logistic Regression with Support Vector Classification, Random Forest against Extra Trees, Gradient Boosting and K-Nearest Neighbors. This model set is good as you can use linear baselines, instance-based learners, kernel and ensemble methods. The aim is not to say that one algorithm has all the time a better performance. What we aim to demonstrate is how these models perform under different assumptions and review metrics. It is presented as follows in the paper.

This section presents research problem, dataset, preprocessing model, selection of the targeted model, conversion strategy of target, imbalance rule and evaluation. The last part, Results, explains how an interpretation of output tables and plots should be used, and why metrics like RMSE or macro F1 are more important to know than a single accuracy value. Conclusion summarizes the contribution of the project and presents potential extensions such as cross-validation, hyperparameter optimization, feature engineering and explainable artificial intelligence.

Dataset Shape: (1460, 81)

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolQC	Fence	MiscFeature	MiscVal	MoSold	YrSold	SaleType	SaleCondition	SalePrice
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	2008	WD	Normal	208500
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...	0	NaN	NaN	NaN	0	5	2007	WD	Normal	181500
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	9	2008	WD	Normal	223500
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	2	2006	WD	Abnorml	140000
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	...	0	NaN	NaN	NaN	0	12	2008	WD	Normal	250000

5 rows x 81 columns

Figure 1: Insights about the dataset and handling missing values

1.1 Project Objectives

The first aim is to construct a fully-fledged regression pipeline for SalePrice prediction. The second goal is to change the regression target into a classification target without fabricating imbalance. The third aim is to evaluate multiple machine learning systems using the same preprocessing conditions. The fourth goal is to test the models with task-appropriate metrics. We require error-based metrics for regression. For classification, metrics need to provide not just the overall correctness of the model but also the behavior of the model over different classes. These objectives make the project potentially suited for a course assignment as they integrate many ideas that are usually taught independently: supervised learning; data cleaning; scaling; encoding; train-test splitting; model selection; evaluation; and imbalanced learning. The project is also an opportunity for discussion. For example, should one only predict the exact house price, or could it more effectively classify the property into a price band? Depending on the application, the answer can be different. A bank may prefer a number. Categories may be preferred by a search platform. A student project could be studying both.

2. Methodology

2.1 Dataset Description

We chose the Ames Housing dataset, as it is big enough to perform significant supervised learning, but still small enough to run in Google Colab. This dataset of the sale of residential properties in Ames, Iowa, contains many of the variables used to evaluate house values. Unlike more dated educational datasets, Ames Housing also qualifies as a candidate for a contemporary preprocessing task as it has a combination of numerical, ordinal, and nominal predictors, as well as missing values. This version employed in the notebook conforms to the popular Kaggle training file, utilizing SalePrice as the target variable and the other variables being predictors. The project code directly extracts the dataset using publicly available CSV mirrors. This style prevents manual upload and allows the notebook to be run in Colab end to end. As external servers can occasionally disconnect, the code attempts a few links. If one link fails, a second one is employed. Thus, the implementation is more stable than one OpenML request. Upon loading, the Id column is discarded since this is an identifier, not a meaningful predictor. The target column is given numeric values and rows with missing SalePrice are excluded. The other predictors are automatically split into numeric and categorical columns for the types of our data.

2.2 Research design

The research design is basic, however, disciplined from a machine learning point of view. The dataset is first loaded and inspected. Second, the predictors are kept distinct from the target variable. Third, the data is split into training and test sets. Fourth, preprocessing is fitted on the training data. Fifth, multiple models are trained and evaluated using the same test set. In the classification part, all of the predictors are the same ones, just that the target is re-structured which is changed from continuous sale price into 3 classes. Then before any imbalance correction is applied, the class distribution is checked. This design makes a difference because it preserves fairness in the experiment. If one model gets scaled features and the other models get raw values, the comparison is weak. If missing values are handled differently for each model, the final ranking may be a result of preprocessing decisions, rather than the overall quality of the model. The

notebook has worked around this by placing the preprocessing in the context of the reusable pipeline. The estimator is changed but the cleaning and transformation logic hasn't changed.

2.3 Data Preprocessing

There are two branches in the preprocessing stage. Median imputation is applied on the numerical columns. Median imputation can be used on housing data because factors such as lot area, basement area, garage size, etc., may be skewed. The median is less impacted by extreme values than the mean. Numerical variables are scaled following imputation by RobustScaler. This scaler applies median- and interquartile range-based statistics so it is less sensitive to large outlier data than conventional z-score scaling. This is helpful with housing data, where some houses have unusually large areas or high sale prices. Categorical columns are populated using most-frequent imputation, then one-hot encoding. Most-frequent imputation replaces missing labeling with the most common category in that feature. Next, after this, one-hot encoding reduces each category into binary indicator columns. This is important since the majority of scikit-learn estimators do not directly process text labels. The encoder has been designed to ignore unknown categories for prediction and thus avoid errors if a category appears in the test set but not in training. This small detail gives robustness.

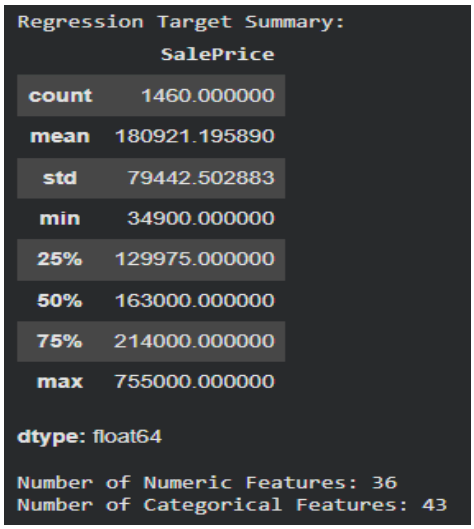


Figure 2: Preprocessing techniques applied to the regression dataset

2.4 Regression task

The regression task predicts the continuous SalePrice value. Before model training, the target is transformed using log1p inside TransformedTargetRegressor. House prices are usually right-skewed because most properties fall within a middle range while a smaller number are very expensive. A logarithmic transformation can reduce this skew and help models focus on relative error patterns. After prediction, the inverse transformation expm1 returns the estimates to the original price scale. This means the final metrics are still reported in interpretable price units. Seven regression models are compared. Linear Regression is included as a baseline. Ridge Regression adds L2 regularization, which can reduce instability when one-hot encoding creates many correlated features. K-Nearest Neighbors predicts based on nearby examples. Support Vector Regression uses a kernel to model non-linear relationships. Random Forest and Extra Trees use ensembles of decision trees, while Gradient Boosting builds trees sequentially to correct previous errors. Together, these models provide a reasonable comparison between simple, distance-based, kernel-based, and ensemble-based approaches.

2.5 Classification Task

The second part of the project treats SalePrice as classes. This is so since not all practical systems require an exact number for price. For example, a housing search application might divide property prices into affordable, standard, and expensive tiers. The established thresholds are: SalePrice below 150,000 dollars is labelled affordable; SalePrice from 150,000 to below 250,000 dollars is labeled standard; and SalePrice of 250,000 dollars or more is labeled expensive. These are thresholds, fixed price ranges, not quantile bins. That choice is important. Quantile bins would make classes of the same size and would hide the natural class distribution. This is not an experiment to artificially manage a distribution — we want to look at the data. The code prints class counts, which will eventually show the class and plot the class distribution once the new target is created. It calculates an imbalance ratio by dividing the size of the largest class by the size of the smallest one. It considers a dataset as imbalanced if the ratio is at least 1.5. This threshold will not be a general rule of thumb, but it is a sensible project rule with which to distinguish when special handling is appropriate. The models train normally if the ratio is lower. If it is larger, class weights

are applied where possible, and oversampling is applied to chosen models with lack of inherent support for class weight.

2.6 Evaluation Metrics

Regression and classification calls for different metrics. Project reporting Mean Squared Error, Root Mean Squared Error, Mean Absolute Error, R-squared and Mean Absolute Percentage Error are all for regression. RMSE is especially suited to measure the measurement in the same unit as the target, dollars. MAE is more accessible since it is the average absolute error. R-squared indicates the fraction of variance being explained by the model. MAPE expresses that level as the percentage of the actual sale price error but it should be interpreted carefully, as percentage errors can act bizarrely when the target is small. To classify, accuracy, precision, recall, macro F1, weighted F1, confusion matrix has been reports in the project. Precision: The degree to which a predicted example of a class is correct. Recall: How many true examples of a class there are. Precision and recall are combined to produce F1. In the classification section, Macro F1 is the main model-selection metric because it does not allow the majority class to dominate the score. The confusion matrix adds a practical view by showing how often the classes are confused with each other.

========== REGRESSION RESULTS ==========

	Model	MSE	RMSE	MAE	R2 Score	MAPE %
0	Linear Regression	5.171531e+08	22741.000212	14900.013348	0.932577	9.041221
1	Ridge Regression	6.299334e+08	25098.474633	16379.194387	0.917874	9.837736
5	Extra Trees Regressor	7.813646e+08	27952.900075	17482.991394	0.898132	10.543728
4	Random Forest Regressor	8.387443e+08	28961.081709	17156.996982	0.890651	10.172321
6	Gradient Boosting Regressor	9.428893e+08	30706.502262	16615.732469	0.877073	9.379416
3	SVR	9.598937e+08	30982.150940	18507.443678	0.874856	11.087270
2	KNN Regressor	2.844632e+09	53335.095289	30208.982272	0.629138	17.768174

Figure 3: Showing evaluation metrics and model scores

3. Results

The notebook produces two primary result tables for regression and classification. The regression table is sorted by RMSE, so the model with the smallest RMSE appears first. The classification table is sorted by macro F1, so the model that performs most evenly across the three price classes appears first. This sorting choice corresponds to the goal of each task. So here, smaller error is better for continuous price estimation. For multi-class prediction and especially when imbalanced, balanced class-level performance matters more than just overall accuracy. Figure 3 reports the regression results for all seven models. Linear Regression achieved the lowest error, with an RMSE of about $22,741, an MAE of about $14,900 and an R^2 of 0.933, followed by Ridge Regression (RMSE of about $25,098, R^2 of 0.918). The tree ensembles came next: Extra Trees (RMSE of about $27,953), Random Forest (about $28,961) and Gradient Boosting (about $30,707). SVR (about $30,982) and KNN (about $53,335, R^2 of 0.629) had the highest errors. Figure 4 shows the same ranking visually.

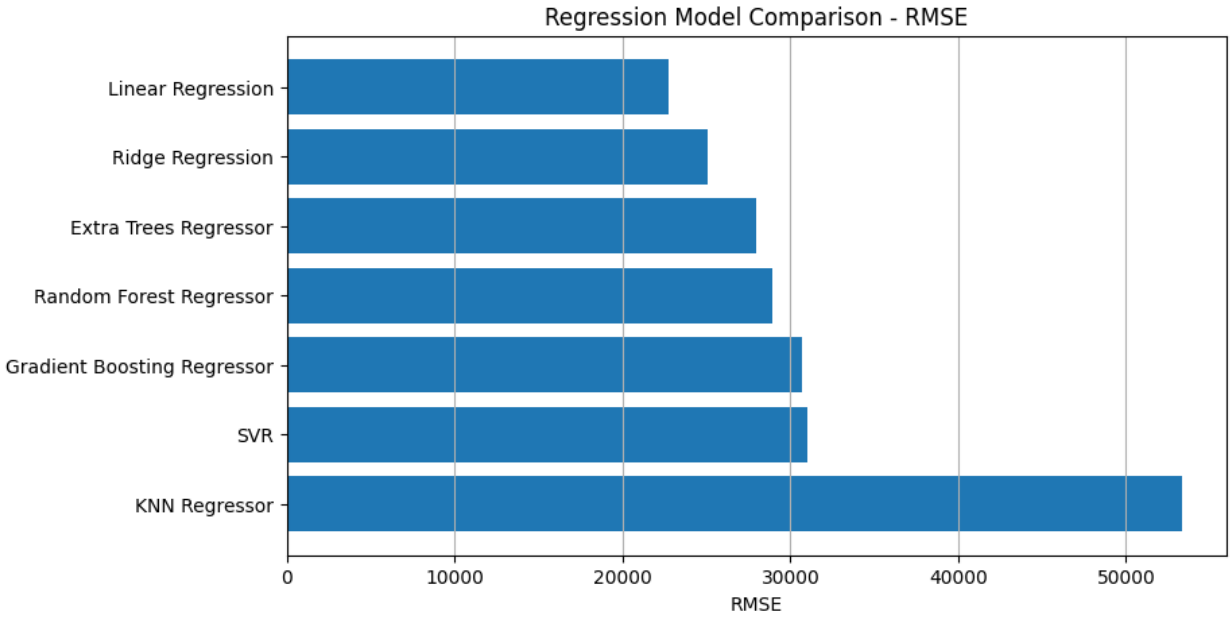


Figure 4: Different model comparison using the regression task

Tree ensembles are often reported to work well on housing data because they can capture non-linear interactions among property characteristics. For instance, the influence of the living area

may depend on neighborhood, building quality, and overall condition. Linear only can give general insights into a relationship, but not interaction (unless the interaction between terms is controlled manually). By contrast, tree ensembles help us with modelling interaction structure more naturally. In this experiment, however, the linear models performed best. A likely reason is that the log transformation of SalePrice, combined with one-hot encoding of the categorical features, makes the relationship between the predictors and the target close to linear. Since no hyperparameter tuning was performed, the ensembles may also improve with tuning. This shows why linear baselines matter: they give a reference point and indicate whether the problem is largely linear or highly non-linear. The interpretation is based on the code's residual plot. A good regression model should have residuals around zero, not a single trend. When the residuals go up for expensive houses you could end up with a less accurate model in the high end of the market. That sort of thing is how we see it in housing data. High-priced properties are frequently more unique, making them harder to predict through general features. The actual-versus-predicted scatter plot helps too. Points close to the diagonal show accurate predictions, points far from them show large errors.

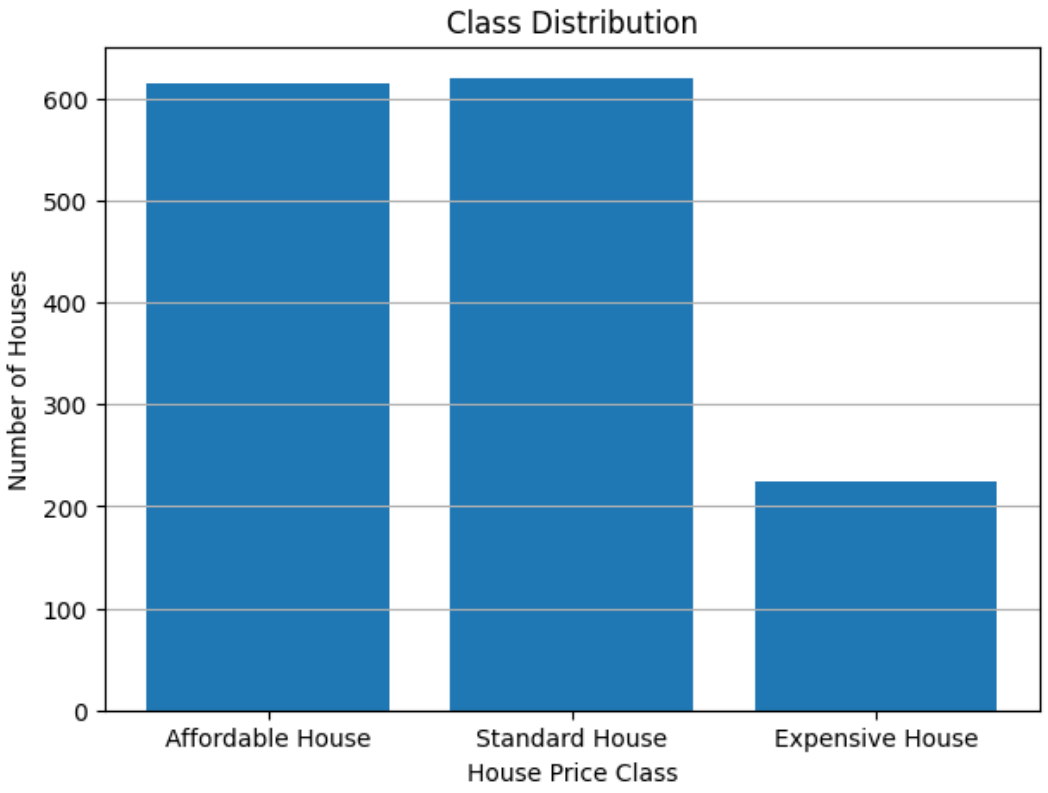


Figure 5: Different class distribution for the classification task showing imbalance

Figure 5 shows that the expensive class is much smaller than the affordable and standard classes (roughly a third of their size), so the imbalance ratio exceeds the 1.5 threshold and imbalance handling was applied. Figure 6 shows the confusion matrix for Gradient Boosting with oversampling, which correctly classified 254 of the 292 test houses (87.0% accuracy). Most errors happen between neighboring price bands: 11 affordable houses were predicted as standard, 16 standard houses as affordable, and 9 expensive houses as standard, while affordable and expensive houses were never confused with each other. The confusion matrix is more informative than accuracy for the classification task. If the affordable and standard classes are often confused, then the threshold between those price ranges may be too close for the available features. The minority high-price class might need stronger handling, better features, or a different threshold if expensive houses are often predicted as standard. Hence the project is not limited to a single score. It uses several outputs together: class counts, imbalance ratio, macro F1, weighted F1, classification report, and confusion matrix. The class distribution plot provides a diagnostic function. If the affordable and standard classes dominate, then a model can learn safe majority-class predictions. When evaluating, Macro F1 weights each class equally, which minimizes this type of issue. It also prints if any imbalance handling was used. It is very transparent and makes the process possible. So the report can explain why class weights or oversampling were used instead of applying them blindly without evidence. Another consequence of the project, however, is methodological rather than purely numerical. The same dataset allows investigation of two supervised-learning paradigms. As a regression task, it asks: how close is the predicted price to the actual price? As a classification task, it asks: can the model place a property into the correct price band? These two questions are related but not the same. In other words, the model can still have moderate regression error while classifying most houses into the “right” price group. On the other hand, a model might predict many prices near the actual values but still fail near class boundaries. And this comparison provides a much more powerful learning component for the project than a single-task experiment.

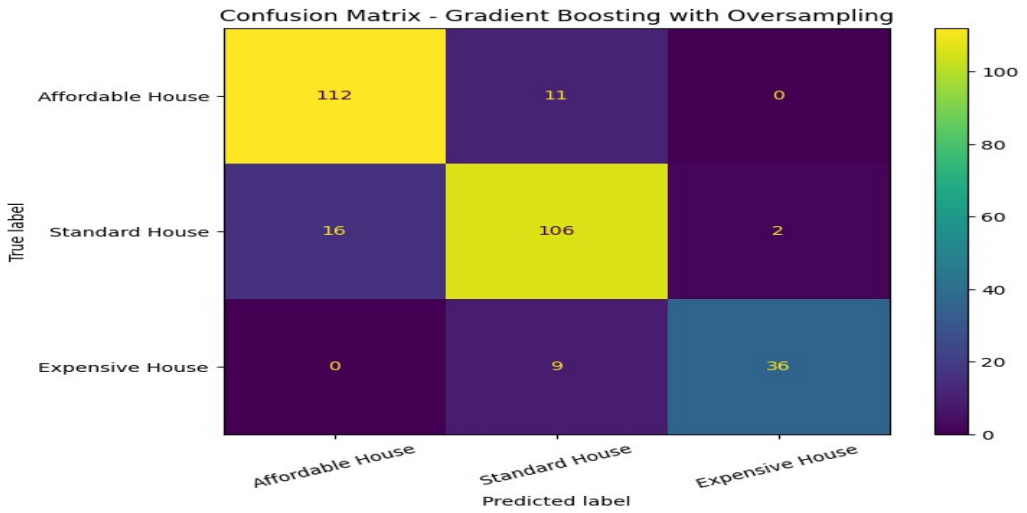


Figure 6: Confusion matrix showing the different classes

4. Conclusion

This work created a full machine learning pipeline for a realistic tabular housing dataset. It began with a regression problem to predict SalePrice from a mix of numeric and categorical predictors. It then converted the target into three meaningful price classes and treated the same dataset as a classification problem. Linear Regression gave the lowest error for price prediction (RMSE of about $22,741, R^2 of 0.933), and Gradient Boosting with oversampling placed 87% of the test houses in the correct price band. The end result is a two-pronged undertaking that bridges multiple key concepts in machine learning: preprocessing, train-test splitting, supervised modeling, model comparison, metric selection, and class-imbalance handling. The strongest part of the project is its pipeline design. Missing values, scaling, and encoding are wrapped within reproducible workflows. This keeps preprocessing consistent and makes data leakage less likely. The code also manages imbalance appropriately. It does NOT establish artificial imbalance. It measures the class distribution after conversion to target, and applies class weights or oversampling if the imbalance ratio indicates such. That's a better method than resampling blindly. The project also demonstrates why evaluation should match the task. Regression depends on error measures like RMSE and MAE. Classification depends on precision, recall, F1 score, and confusion matrices. Accuracy alone is not sufficient when one class may be smaller than another. By employing macro F1 as the classification selection metric, this project enables fairer assessment of minority classes. Several improvements can be made. First, this notebook

could add cross-validation so that the model ranking does not solely hinge on one train-test split. Second, hyperparameter tuning can be done with RandomizedSearchCV or GridSearchCV. Third, feature engineering could be broadened by adding other features, such as house age at sale, total finished square footage, or quality-area interactions. Fourth, explainability tools such as permutation importance or SHAP can be applied to make the model more interpretable. Fifth, classification thresholds could be validated against several business cases. For instance, a real estate website can favor price bands other than those in a bank. Overall, the project focuses on more than training a single model: it constructs an explainable and reusable machine learning process. It also honors the distinction between regression and classification yet demonstrates how the same data can accommodate both. The final notebook runs end to end in Google Colab.

5. References

Chen, H. (2024). House price prediction based on machine learning model. In Proceedings of DESD 2024. Atlantis Press.

De Cock, D. (2011). Ames, Iowa: Alternative to the Boston Housing Data as an end of semester regression project. Journal of Statistics Education, 19(3).

Gong, H. (2025). Comparative analysis of machine learning models for house price prediction. In Proceedings of ICIAAI 2025. Atlantis Press.

Jiang, H. (2025). Machine learning models for predicting second-hand house prices. ACM International Conference Proceedings.

Khan, A. A., et al. (2024). A review of ensemble learning and data augmentation models for class imbalance problems. Expert Systems with Applications.

Kinnunen, T. (2025). House price prediction using model stacking. Master’s thesis, Tampere University.

Koukaras, P., et al. (2025). Data preprocessing and feature engineering for data mining. Data, 6(10), 257.

Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. Journal of Machine Learning Research, 18(17), 1-5.

Mora-Garcia, R. T., et al. (2022). Housing price prediction using machine learning algorithms: The case of the real estate market. Land, 11(11), 2100.

Oh, S. Y. (2024). Prediction of residential property prices using machine learning algorithms. ITM Web of Conferences.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Sharma, H., et al. (2024). An optimal house price prediction algorithm: XGBoost. Analytics, 3.

Taskiran, S. F., et al. (2025). A comprehensive evaluation of oversampling techniques for imbalanced classification. Scientific Reports.

Yang, X. (2025). Research on house price prediction based on machine learning. ITM Web of Conferences.